

HOMework ODS

CISOTTO MARTINA 2182469

GROSSELLE ALICE 2199312

Abstract

This report analyses the optimization of multiclass logistic regression models using two first-order methods: Gradient Descent and Block Coordinate Gradient Descent with the Gauss-Southwell rule.

The study involves the minimization of the negative log-likelihood function for high-dimensional classification problems. We explain the implementation of our algorithms and describe some related topics like the loss calculation, the performance evaluation, and the choice of the learning rate.

The algorithms are analyzed on a random matrix and on the real world ISOLET5 dataset. Results prove that while the BCGD with Gauss-Southwell rule gives higher progress per iteration, selecting the most significant gradient blocks, Gradient Descent remains more efficient in terms of CPU time.

Finally we analyze the performance with loss and accuracy metrics, underlining the trade-offs between theoretical iteration complexity and practical CPU time.

1 Introduction

The goal of this homework is to study the multiclass logistic regression problem by implementing and comparing two optimization methods. The task consists in finding a weight matrix that minimizes the negative log-likelihood of a dataset, mapping input features to a probability distribution over multiple classes. This type of problem is common in supervised learning, and some applications such as speech recognition.

To build a reliable implementation, we first develop a set of core functions for probability computation and loss calculation. Since the softmax function involves exponential terms, we try to avoid numerical issues such as overflow or underflow. For this reason, we use simple stabilization techniques to ensure that the computations remain accurate even on larger datasets.

We then analyze two different optimization strategies. The first is Gradient Descent (GD), which updates all parameters at each step using the full gradient. The second is Block Coordinate Gradient Descent with the Gauss-Southwell rule (BCGD-GS), which updates only one block of parameters at a time, selecting the one that has the largest impact on the objective function.

This comparison highlights the trade-off between per-iteration progress and computational cost. While BCGD-GS may achieve larger improvements at each step, it requires additional work to select the block to update. We evaluate both methods on random data and on the real-world ISOLET dataset, focusing on convergence behavior and execution time. The final analysis examines how these factors influence the overall efficiency and stability of the optimization process.

2 Formulation

This report focuses on the implementation and analysis of a multiclass logistic regression problem, a supervised learning task where the goal is to classify data points into one of the k distinct categories. Given a set of m training samples with features $a_i \in \mathbb{R}^d$ and the corresponding labels $b_i \in 1, \dots, k$, we model the probability of a label b_i using the Softmax function:

$$P(b_i|a_i, X) = \frac{\exp(x_{b_i}^\top a_i)}{\sum_{c=1}^k \exp(x_c^\top a_i)} \quad (1)$$

where X represents the weight matrix to optimize.

To find the optimal parameters we need to maximize the likelihood over m independent and identically distributed samples, which is equivalent to minimizing the negative log-likelihood function:

$$\hat{f}(X) = \sum_{i=1}^m \left[-x_{b_i}^\top a_i + \log \left(\sum_{c=1}^k \exp(x_c^\top a_i) \right) \right] \quad (2)$$

The function is smooth and its partial derivative with respect to the parameters is defined as:

$$\frac{\partial f(X)}{\partial x_c} = \sum_{i=1}^m a_i \left[\frac{\exp(x_c^\top a_i)}{\sum_{j=1}^k \exp(x_j^\top a_i)} - I(b_i = c) \right] \quad (3)$$

where I is the indicator variable, which can be 1 if $b_i = c$ and 0 otherwise.

In the following sections, we implement and compare two optimization methods: Gradient Descent and Block Coordinate Gradient Descent with the Gauss-Southwell rule. We test these algorithms on both the ISOLET5 dataset and on randomly generated data.

ISOLET Dataset The real world application involves the ISOLET5 dataset, a part of the Isolated Letter Speech Recognition database. The implementation applies a multiclass classifier able to distinguish the letters of the English alphabet based on spoken audio.

- Target classes (k): 26, one for each letter of the alphabet
- Feature space (d): 617 numerical features. These include spectral coefficients, intensity contours, and phonetic characteristics extracted from the raw audio
- Sample size (m): 1,559 observations, representing the final portion of the original study (ISOLET5)

The matrix $A \in \mathbb{R}^{m \times d}$ is standardized to have mean 0 and standard error 1. To improve numerical stability, we use `StandardScaler()` to center the features. This step is important for multiclass logistic regression, as it helps to make the computation of the Lipschitz constants more reliable.

The algorithms aim to find the optimal weight matrix $X \in \mathbb{R}^{617 \times 26}$ that minimizes the multi classification error. The matrix A is splitted into training and test sets, in this way we can train our model only in a part of the dataset and then try the methods and evaluate the performance on new data.

Random Data The functions are also applied to randomly generated data to validate the mathematical convergence and performance of the methods.

- The random feature matrix A is generated using a standard normal distribution $N(0, 1)$ with dimensions 1000×1000 ($m = 1000, d = 1000$).

- The class labels b are created by computing a linear transformation $AX + E$, where E represents the Gaussian noise. The class labels are assigned based on the maximum index of the resulting rows.

By applying our Gradient Descent and BCGD with Gauss-Southwell implementations to the dataset ISOLET and the random matrices, we can effectively analyze the convergence trends of each approach.

The implementation of optimization algorithms focuses on three fundamental functions necessary for multiclass logistic regression: probability estimation, loss calculation, and performance evaluation.

All functions use the NumPy library for fast and efficient vectorized operations.

2.1 Probability Estimation

The `compute_prob` function computes the class probabilities for each observation using the Softmax activation function. It first calculates the score matrix $Z \in \mathbb{R}^{m \times k}$ through the dot product:

$$Z = AX \quad (4)$$

where A is the feature matrix and X is the weight matrix. Each element $x_c^\top a_i$ represents the score for sample i and class $c \in \{1, \dots, k\}$.

To ensure numerical stability and prevent overflow errors with large exponential values, the implementation uses a stabilization technique: subtract the maximum value for every row, $z_i^{max} = \max_c(x_c^\top a_i)$ from the logits. This is implemented using `np.max(Z, axis=1)` to find the maximum in the class.

This normalization ensures that the predicted probabilities for each observation are between 0 and 1 and sum to 1, matching the model likelihood.

The probability that a sample i belongs to class c is then calculated as:

$$P(c|a_i, X) = \frac{\exp(x_c^\top a_i - z_i^{max})}{\sum_{j=1}^k \exp(x_j^\top a_i - z_i^{max})} \quad (5)$$

In this way the numerator for each row is calculated by the exponential of the subtraction between our score Z and the maximum value of it, while the denominator is the sum of all the computed numerators.

2.2 Loss Calculation

The objective of the `compute_loss` function is to calculate the negative log-likelihood of the model, which is the cost function $f(X)$ that needs to be minimized.

The loss is computed using a stabilized formulation to maintain numerical stability, rather than computing the Softmax directly. We use the average loss to ensure that the objective function remains independent of the number of samples (m), this way the gradient scale does not grow linearly with the size of the dataset, making simpler the selection of an appropriate learning rate. This choice is consistent with the theoretically derived Lipschitz constant $L_{tot} = \sum A_{ij}^2/4m$, that includes the $1/m$ term, so that the step size $\alpha = q/L$ is scaled, maintaining stability independently of the dataset size.

The average loss for m samples is defined by the following formulation:

$$f(X) = \frac{1}{m} \sum_{i=1}^m \left[\underbrace{-x_{b_i}^\top a_i}_{\text{Term 1}} + \underbrace{\log \left(\sum_{c=1}^k \exp(x_c^\top a_i) \right)}_{\text{Term 2}} \right] \quad (6)$$

We know that every element of Z is computed by the dot product between the matrices A and X , in particular $z_i = x_c^\top a_i$.

The first term `T_1` selects the calculated score for the correct class b_i for the i -th observation. In the implementation, it is computed efficiently using NumPy's advanced indexing: `Z[np.arange(m), b]`.

The second term `T_2` is obtained by using the following form to ensure numerical stability:

$$\log \left(\sum_{c=1}^K \exp(z_{ic}) \right) = \log \left(\sum_{c=1}^K \exp(z_{ic} - z_i^{\max}) \right) + z_i^{\max} \quad (7)$$

This formulation computes the logarithm of the sum of exponentials, in which for every element of Z the maximum score is subtracted, and then it sums the maximum subtracted previously to ensure the correct result.

This approach avoids two computational problems: the overflow, because it prevents that large positive values in Z result in infinite values during exponentiation, and the underflow, because if a probability is very close to zero, taking its logarithm would result in $-\infty$, stopping the Gradient Descent process.

By calculating the log-probabilities through these combined terms, the function provides a robust scalar value representing the model's average error across the training set.

2.3 Performance evaluation

The `compute_accuracy` function evaluates the performance of our model calculating the accuracy score. The previous functions are used for training (calculating probabilities and loss), but this function is used for testing, to see how many predictions the model makes right.

The formula is the following:

$$\text{Accuracy} = \frac{1}{m} \sum_{i=1}^m I(\hat{y}_i = b_i) \quad (8)$$

where m is the total number of samples, $\hat{y}_i$ is the predicted class (`predict`), b_i is the true class, and I is the indicator function, which returns 1 if correct and 0 if wrong.

In the implementation, the predicted class $\hat{y}_i$ is obtained by the index with the highest score for each observation through the `np.argmax(Z, axis=1)` function. This index represents the class for which the model has the highest confidence. The final accuracy is computed as the mean of `predict == b`, that calculates the ratio of correct predictions over the total number of observations.

3 Algorithms

To solve the multiclass logistic regression problem, we implement and compare two first-order optimization methods: Gradient Descent (GD) and Block Coordinate Gradient Descent with the Gauss-Southwell rule (BCGD-GS).

Both approaches are appropriate for this minimization problem because the objective function, defined as the negative log-likelihood, is smooth and convex, guaranteeing that any local minimum is a global minimum.

Gradient Descent updates the entire weight matrix X simultaneously by moving in the direction of the steepest descent. BCGD with Gauss-Southwell rule divide the optimization into blocks and it targets the block with the largest gradient norm, useful for high dimensional data where some features may contribute more to the total error than others.

In the following algorithms we choose some parameters, such as `iters` and `eps` for the stopping condition. We also implement a grid search to choose q for the best learning rate for our datasets, computed also considering Lipschitz constant.

Convergence The execution of both algorithms is controlled by two main parameters: `max_iter` and `eps`.

- Iteration limit: the `max_iter` parameter sets the maximum number of iterations the algorithms are allowed to do. This is a safety limit to ensure the algorithm terminates in a reasonable time, even if the best accuracy is not reached.
- Stopping condition: to make sure the algorithm stops when it reaches a stationary point, where other updates will not lead to significant improvements, we use the Frobenius norm of the full gradient matrix, for both methods. The process stops when the norm is smaller than the threshold ϵ (`eps`):

$$\|\nabla f(X)\|_F = \sqrt{\sum_{j=1}^d \sum_{c=1}^k (\nabla f(X)_{jc})^2} < \epsilon \quad (9)$$

This ensures that the optimization ends when the partial derivatives for all features and classes are sufficiently close to zero, this means that the model has successfully converged to an optimal solution.

Grid Search An important aspect of our implementation is the choice of the learning rate α .

In theory the stability of the algorithms is guaranteed by L . To speed up convergence while maintaining stability, in our code we define the learning rate as $\alpha = q/L$, where q is an hyperparameter and L is the global Lipschitz constant of the gradient (in the next sections the choice of the learning rate will be explained in detail for both algorithms).

To find the optimal value for q , we implemented the `grid_search_q` function. This function search on a set of values `possible_q`.

This approach ensures that the learning rate is mathematically grounded in the smoothness of the objective function, while q allows for empirical fine-tuning to accelerate convergence. By testing a range of values over a fixed number of iterations, the function identifies the q that maximizes the accuracy of the test. This process effectively balances the theoretical constraints of the Lipschitz constant with practical performance, ensuring a faster and more robust convergence on the ISOLET dataset.

3.1 GRADIENT DESCENT

Gradient Descent is an optimization algorithm that updates the entire weight matrix $X \in \mathbb{R}^{d \times k}$, where d is the number of features and k is the number of classes, simultaneously by moving in the direction of the steepest descent of the loss function.

Unlike stochastic or coordinate approaches, GD uses the information from the entire dataset to perform a single and stable update per iteration.

3.1.1 Gradient computation

In our code, the function `compute_prob` calculates the probabilities called $S \in \mathbb{R}^{m \times k}$.

The gradient of the multinomial logistic loss is then computed as:

$$\nabla f(X) = \frac{1}{m} A^T (S - Y) \quad (10)$$

where Y is the `indicator` matrix (one-hot encoded labels). This is translated into the code using the operation `np.dot(A.T, (S - indicator)) / m`.

The `indicator` matrix $Y \in \{0, 1\}^{m \times k}$ is the ground-truth target for the multiclass classification task. In the implementation, it is built using one-hot encoding, where for each sample i , the element $Y_{i,c} = 1$ if the true label is c , and 0 otherwise. This matrix is important for computing the error term $(S - Y)$, which measures the difference between the predicted probabilities and the true class labels. By constructing this matrix before the optimization loop, the algorithm can compute the full gradient vectorized. This allows efficient weight updates and helps minimize cross-entropy loss over all classes at the same time.

3.1.2 Learning rate and global Lipschitz constant

For a stable convergence, we use a global Lipschitz constant L_{tot} . This implementation is derived from the squared Frobenius norm of the data matrix A (as an upper bound to the spectral norm) to ensure a safe learning rate for the entire gradient:

$$L_{tot} = \frac{\sum A_{ij}^2}{4m} \quad (11)$$

The Lipschitz constant L_{tot} represents the maximum rate of change of the gradient, providing a theoretical upper bound on the curvature of the objective function. The choice of the factor $1/4$ in the denominator is from the mathematical properties of the multinomial logistic loss. The second derivative, the curvature of the softmax function is bounded by 0.25. By combining this bound with the data matrix A and normalizing by the number of samples m , we have a theoretical Lipschitz constant. This value is the maximum safe step length: a step size that is too large could cause the algorithm to diverge, while one based on the Lipschitz constant guarantees that the step size α is small enough to prevent divergence while remaining large enough to allow steady progress toward the minimum.

The learning rate α remains constant across all iterations and is modulated by a scaling factor q :

$$\alpha = q/L_{tot} \quad (12)$$

In this way we ensure that the update does not overshoot the local minimum.

3.1.3 Weight matrix update

The principal characteristic of this Gradient Descent implementation is the simultaneous update of the entire weight matrix X . Every feature's weight in all classes is modified in a single step using the full gradient $\nabla f(X)$. Mathematically the update logic follows these steps:

1. **Direction:** The update direction is defined by the full gradient matrix $\nabla f(X) \in \mathbb{R}^{d \times k}$, representing the steepest slope direction for all parameters simultaneously.
2. **Learning rate:** The step length of the update is controlled by the global learning rate α . The resulting step applied to the weights is $\alpha \nabla f(X)$.

Algorithm 1 Gradient Descent

Require: Training data $A \in \mathbb{R}^{m \times d}$, labels $b \in \{1, \dots, K\}^m$, number of classes k , learning rate factor q , tolerance ϵ , max iterations I

Optional: Test data A_{test} , test labels b_{test} to compute accuracy on dataset

Ensure: Optimized weight matrix $X \in \mathbb{R}^{d \times k}$, history

```
1: Initialize  $X^{(0)} \leftarrow \mathbf{0}_{d \times k}$ 
2: Construct indicator matrix  $Y \in \{0, 1\}^{m \times K}$  where  $Y_{i,b_i} = 1$ 
3: Compute Lipschitz constant:  $L_{tot} \leftarrow \frac{\sum A_{i,j}^2}{4m}$ 
4: Set learning rate:  $\alpha \leftarrow q/L_{tot}$ 
5: for  $i = 0, 1, \dots, I - 1$  do
6:   Compute probabilities:  $S \leftarrow \text{compute\_prob}(X^{(i)}, A)$ 
7:   Compute full gradient matrix:  $\nabla f(X^{(i)}) \leftarrow \frac{1}{m} A^\top (S - Y)$ 
8:   Compute full gradient norm:  $G_{norm} \leftarrow \|\nabla f(X^{(i)})\|_F$ 
9:   if  $G_{norm} < \epsilon$  then ▷ Stopping condition
10:    break
11:   end if
12:   Update weight matrix:  $X^{(i+1)} \leftarrow X^{(i)} - \alpha \nabla f(X^{(i)})$ 
13:   Save history: duration, loss, and test accuracy
14: end for
15: return  $X^{(i+1)}$ , history
```

3. **Matrix Update:** The new weight matrix is computed by subtracting this update from the current state:

$$X^{(new)} = X^{(old)} - \alpha \nabla f(X) \quad (13)$$

This "all-at-once" update reflects the batch nature of the algorithm. The function controls convergence by checking if the Frobenius norm of the gradient (**grad_norm**) is lower than a threshold **eps**, allowing stopping before reaching the maximum number of iterations (**iters**).

3.2 BCGD WITH GAUSS-SOUTHWELL RULE

The Block Coordinate Gradient Descent with the Gauss-Southwell rule is a greedy optimization strategy. In this implementation, each "block" corresponds to a row of the weight matrix $X \in \mathbb{R}^{d \times k}$, representing the weights of a single feature across all classes. Instead of updating all parameters simultaneously, the algorithm selects the feature j^* that has the largest gradient norm to ensure the maximum possible functional decrease at each step.

3.2.1 Gradient computation and block selection

Similarly to the Gradient Descent implementation, the function **compute_prob** calculates the probability matrix $S \in \mathbb{R}^{m \times k}$. The full gradient, in the code called **grad_matrix**, is computed as:

$$\nabla f(X) = \frac{1}{m} A^\top (S - Y) \quad (14)$$

To apply the Gauss-Southwell rule, the algorithm calculates the L_2 norm of each row of the gradient matrix, saved in the vector **row_norms**, with $r_j = \|\nabla f(X)_{j,:}\|_2$. The index of the row to be updated, which corresponds to the variable **max_row** in our implementation, is selected as:

$$j^* = \operatorname{argmax}_{j \in \{1, \dots, d\}} r_j \quad (15)$$

This selection identifies the feature with the largest error, allowing the algorithm to focus on the direction that reduces the loss the most.

3.2.2 Learning rate and local Lipschitz constants

Unlike GD, BCGD uses local Lipschitz constants L_j for each block, computed at the beginning in the array `L_blocks`. These constants are calculated from the squared norm of the columns of the data matrix A :

$$L_j = \frac{\sum_{i=1}^m A_{ij}^2}{4m} \quad (16)$$

The local constant L_j provides a tighter bound on the curvature along the j -th block. The learning rate α for the selected block is then scaled by the factor q :

$$\alpha = q/L_{j^*} \quad (17)$$

In the code, this is implemented as `alpha = q / L_blocks[max_row]`, local constants allow for a more adaptive learning rate, ensuring a step size adapted to the specific curvature of the selected feature.

Algorithm 2 Block Coordinate Gradient Descent with Gauss-Southwell rule

Require: Training data $A \in \mathbb{R}^{m \times d}$, labels $b \in \{1, \dots, k\}^m$, number of classes k , learning rate factor q , tolerance ϵ , max iterations I

Optional: Test data A_{test} , test labels b_{test} to compute accuracy on dataset

Ensure: Optimized weight matrix $X \in \mathbb{R}^{d \times k}$, history

```

1: Initialize  $X^{(0)} \leftarrow \mathbf{0}_{d \times k}$ 
2: Construct indicator matrix  $Y \in \{0, 1\}^{m \times k}$  where  $Y_{i,b_i} = 1$ 
3: Compute block Lipschitz constants:  $L_j \leftarrow \frac{\sum_{i=1}^m A_{i,j}^2}{4m}$  for  $j = 1, \dots, d$ 
4: for  $i = 0, 1, \dots, I - 1$  do
5:   Compute probabilities:  $S \leftarrow \text{compute\_prob}(X^{(i)}, A)$ 
6:   Compute full gradient matrix:  $\nabla f(X^{(i)}) \leftarrow \frac{1}{m} A^\top (S - Y)$ 
7:   Compute row norms:  $r_j \leftarrow \|\nabla f(X^{(i)})_{j,:}\|_2$  for  $j = 1, \dots, d$ 
8:   Compute full gradient norm:  $G_{norm} \leftarrow \sqrt{\sum_{j=1}^d r_j^2}$ 
9:   if  $G_{norm} < \epsilon$  then ▷ Stopping condition
10:     break
11:   end if
12:   Block Selection (Gauss-Southwell rule):  $j^* \leftarrow \arg \max_j r_j$ 
13:   Set block learning rate:  $\alpha \leftarrow q/L_{j^*}$ 
14:   Update selected row:  $X_{j^*,:}^{(i+1)} \leftarrow X_{j^*,:}^{(i)} - \alpha \nabla f(X^{(i)})_{j^*,:}$ 
15:   Keep other rows:  $X_{j,:}^{(i+1)} \leftarrow X_{j,:}^{(i)}$  for all  $j \neq j^*$ 
16:   Save history: duration, loss, and test accuracy
17: end for
18: return  $X^{(i+1)}$ , history

```

3.2.3 Weight matrix update

The fundamental characteristic of this BCGD implementation is that it updates the weights of only one feature at a time across all classes. The approach follows this selective logic:

1. **Selection:** The algorithm finds the index j^* corresponding to the row of the gradient matrix with the highest L_2 norm, making sure that the update is done on the feature with the largest impact on the error.
2. **Learning rate:** The step length of the update is controlled by the local learning rate $\alpha = q/L_{j^*}$. This results in a update vector $\alpha \nabla_{j^*} f(X)$ that is specific for the selected feature, ensuring that the step is appropriate for its specific curvature.
3. **Row update:** Unlike Gradient Descent, which modifies the entire matrix, this step performs a selective update. The new weight matrix is computed by modifying only the j^* -th row:

$$X_{j^*,:}^{(new)} = X_{j^*,:}^{(old)} - \alpha \nabla_{j^*} f(X) \quad (18)$$

In the implementation, this is achieved by slicing the matrix (`X[max_row, :]`), ensuring that all other rows $j \neq j^*$ remain unchanged. This process focuses the optimization to a single feature's weights across all classes.

3.2.4 Computational cost and trade-offs

A key aspect of this implementation is the cost per iteration. To apply the Gauss-Southwell rule, the code computes the full gradient $\nabla f(X)$ at each step to find the maximum row norm.

Time complexity Each iteration costs $O(m \cdot d \cdot k)$ due to the matrix product `np.dot(A.T, S-Y)`. This makes a single BCGD iteration as expensive as a full GD iteration in terms of CPU time.

Efficiency Since BCGD updates only $1/d$ of the parameters per iteration using the same amount of information as GD, it is generally less efficient on dense datasets like ISOLET. The benefit of coordinate descent is normally in the number of updates to reach convergence, but the cost per iteration of the Gauss-Southwell rule is the main problem.

4 Results

In this section the results obtained by testing the Gradient Descent and the Block Coordinate Gradient Descent with the Gauss-Southwell rule are presented. The algorithms are evaluated on a random dataset and the ISOLET dataset. The goal is to analyze the trade-offs between convergence speed, computational effort (CPU time) and accuracy, with a focus on how the learning rate factor q influences the performance.

4.1 Performance on ISOLET dataset

The ISOLET dataset is a real world scenario with a high number of features and 26 classes. We used it to compare the two algorithms using the best learning rate factor q found during the grid search, based on the highest accuracy.

4.1.1 Accuracy vs CPU time analysis

The first metric analyzed is the change of the test accuracy relative to the CPU time. This is an important metric in practical applications, because we are interested in how fast a model becomes accurate, and it is a realistic measure of efficiency. Looking at the plot "Accuracy vs CPU time" (Figure 1), we can observe some trends:

- **Efficiency and convergence speed** Gradient Descent (blue line) shows a really steeper initial curve (over 80% of accuracy almost immediately). With the optimized $q = 10$ it reaches the stopping criterion ($\epsilon = 10^{-1}$) in only 173 iterations, taking 0.28 seconds to achieve a final test accuracy of 94.87%. This is because a single iteration of GD updates all parameters simultaneously, giving an higher accuracy at the beginning. The BCGD-GS algorithm (orange line) starts much lower and takes more time to reach this accuracy.
- **Impact of the selection rule** The BCGD-GS algorithm reaches a similar accuracy (94.23%), but needs many more iterations (1,222) and more time (1.93 seconds) to converge when using $q = 1$. The problem for BCGD-GS in this implementation is the Gauss-Southwell selection rule: we compute the full gradient (a really expensive process) to find the coordinate with the maximum norm, that makes the cost per iteration comparable to a full GD step, but with only one row of the weight matrix X updated. This is less efficient than using that same gradient to update the entire matrix, like GD.
- **Final accuracy and stability** Both algorithms stabilize around a very high accuracy (approximately 94%). This confirms that both methods are minimizing the negative log-likelihood and finding a good solution for the multiclass logistic regression problem. The GD curve is smoother, while the BCGD-GS curve shows slight "staircase" patterns. This is normal for coordinate descent methods where the progress per individual step is much smaller, giving a fragmented improvement in performance.

In summary, for the ISOLET dataset, the Gradient Descent appears to be the best choice when considering the balance between accuracy and CPU time. While the Gauss-Southwell rule is theoretically powerful because it chooses the most important update, the computational cost of the selection process makes it less competitive in our implementation.

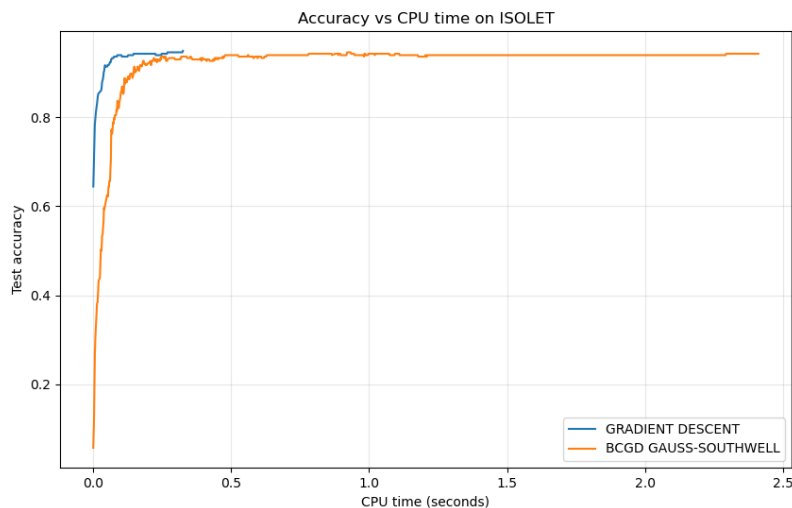


Figure 1: Accuracy vs CPU time in ISOLET

4.1.2 Loss vs CPU Time Analysis

The evolution of the loss function is fundamental to understand the optimization process. While accuracy tells us how many samples are correctly classified, the loss function describes the objective minimization efficiency.

Looking at the plot "Loss vs CPU time on ISOLET" (Figure 2), we can observe a behavior similar to the accuracy results:

- **Rate of minimization** The Gradient Descent loss decreases fast at first. This indicates that the global step updates all $d \times k$ parameters simultaneously, moving the entire model toward the minimum very efficiently. In contrast, the BCGD-GS curve (orange) is slower, requiring more time to reach the same loss levels.
- **Asymptotic behavior** Both algorithms show a classic convex convergence shape, where the loss decreases rapidly at first and then flattens as they arrive near the optimum. However, the difference between the two curves is evident: the GD reaches faster any lower value of loss reached than the BCGD-GS.
- **Efficiency per second vs efficiency per iteration** These are two different concepts. Theoretically, the Gauss-Southwell rule is greedy and choose the update that guarantees the most significant decrease in loss per iteration. However, if we look at the CPU time, there is not this theoretical advantage, because one iteration of BCGD-GS is almost as expensive as one iteration of GD (since both require the full gradient), but BCGD-GS only updates $1/d$ -th of the parameters. As shown in the outputs, GD with an optimized $q = 10$ converges in just 0.28 seconds (173 iterations). BCGD-GS, even if it is effective, requires 1.95 seconds (1,222 iterations) to reach the stopping criterion at $q = 1$. This proves that for the ISOLET dataset, the global updates of GD are almost 7 times faster than the coordinate updates of BCGD-GS in terms of CPU time.

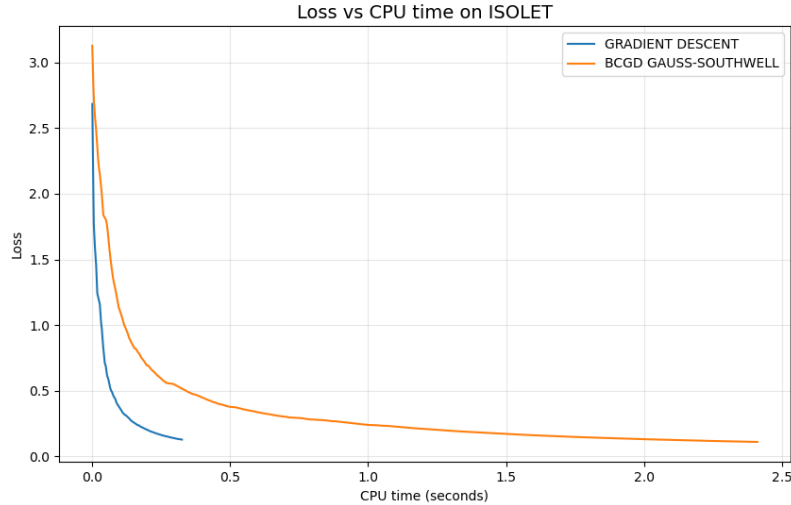


Figure 2: Loss vs CPU time in ISOLET

4.2 Results on Random data

We also test our algorithms on a random dataset, where $m = 1000$ observations and $d = 1000$ features are generated from a standard normal distribution $\mathcal{N}(0, 1)$. This allows to observe the behavior of the algorithms when features are independent and identically distributed.

4.2.1 Convergence and computational cost

Analysing the plots "Loss vs CPU time on Random data" (Figure 3a) and "Loss vs Iterations on Random data" (Figure 3b), it is possible to observe the optimization progress in a high dimensional setting. In this controlled environment, we set a common learning rate factor $q = 1$ for both algorithms to allow a fair comparison:

- Similarity between CPU time and iterations** In this experiment, the plots for Loss vs. Iterations and Loss vs. CPU time have almost the same shape. This is because the computational cost of each iteration is very stable. With random data, the cost of the Gauss-Southwell selection rule is perfectly proportional to the update step, making the two metrics interchangeable.
- Algorithms performance and comparison** On the synthetic dataset the BCGD-GS algorithm (orange line) shows a slightly better convergence rate than the Gradient Descent. While GD updates all parameters at once, BCGD-GS concentrates on the row of the parameter matrix X with the largest gradient norm. This greedy approach is more effective in a random environment ($d = m$), as the Gauss-Southwell rule identifies the best directions for optimization. BCGD-GS, at the end of the 2,000 iterations, reaches a lower loss compared to GD, demonstrating that coordinate methods can be better than full-gradient methods when features have similar scales and the selection rule successfully finds coordinates with highest descent potential.

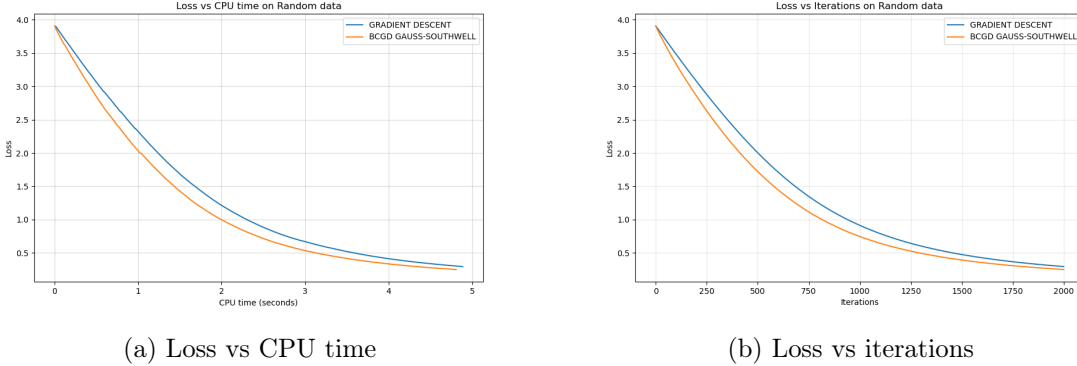


Figure 3: Loss analysis in Random data

4.3 Learning rate sensitivity (q)

An important part of the analysis focuses on the factor q used in the learning rate. The results (Table 1) show that the choice of q strongly affects the performance, especially for the BCGD method.

Gradient Descent Increasing q from 1 to 10 makes convergence much faster. The number of iterations drops from 1,734 to 173, and the running time decreases from 2.63 seconds to 0.28 seconds. The accuracy also improves slightly (94.87%). This suggests that the theoretical learning rate used for GD ($L_{tot} = \sum A^2/4m$) is quite conservative, and larger steps can significantly speed up optimization without harming performance.

BCGD with Gauss-Southwell rule Has a very different behavior. Although it performs well with $q = 1$, it struggles when $q = 10$. The algorithm does not converge within the maximum number of iterations (2,000), and the final accuracy drops to 64.74%, much lower than the 94.23% obtained with $q = 1$. This happens because BCGD-GS uses on local Lipschitz constants for each block update. If q is too large, the update on a single block may overshoot the minimum. Even if a large learning rate works well for a full gradient update, it can lead to instability when updates are performed to blocks that have a higher local curvature.

In summary, for the ISOLET dataset, Gradient Descent with a high learning rate factor ($q = 10$) provides the best balance of speed and precision, reaching accuracy much faster than BCGD.

Table 1: Performance comparison on ISOLET ($\epsilon = 10^{-1}$)

Algorithm	q	Iterations	Time (s)	Acc (%)
GD	1	1734	2.63	94.55
BCGD-GS	1	1222	1.95	94.23
GD	10	173	0.28	94.87
BCGD-GS	10	2000*	3.19	64.74

**For $q = 10$ BCGD-GS did not reach convergence in the `max_iter` limit*

4.4 Conclusions

The results show that the performance of the two methods depends not only on the algorithm and implementation, but also on the data structure and on the choice of the learning rate.

In the **ISOLET** dataset, GD turns out to be the most efficient method in terms of execution time. Thanks to the complete update step, it reaches high accuracy (94.87%) very quickly with the best learning rate factor q for this case. BCGD-GS achieves similar accuracy for smaller values of q , but is slower. This is mainly due to the cost of the Gauss-Southwell rule, which requires computing the full gradient at each iteration while updating only one block.

In the **Random** dataset, the behavior is different. With these more balanced data, BCGD-GS shows faster progress per iteration and reaches a lower final loss than GD. This confirms that selecting coordinates in a greedy way can be effective when the features have similar scales and the problem is more regular.

The experiment on the learning rate factor q finds a trade-off between speed and stability. GD is quite robust and can benefit from larger step sizes, which significantly reduce the number of iterations without losing accuracy. In contrast, BCGD-GS is much more sensitive: large values of q can lead to unstable updates and poor performance.

Finally, BCGD with Gauss-Southwell rule can be a useful method in high dimensional problems where full updates are too expensive, but with our implementation GD is the more efficient. Other alternatives could focus on reducing the cost of the selection step, for example by using randomized rules.